Discussion 4: Scheduling

March 6, 2026

Contents

1 Scheduling

Scheduling is the process of deciding which threads are given access to resources from moment to moment. Usually, scheduling pertains to CPU access times, but it can encompass any type of resource like network bandwidth or disk access.

When discussing scheduling, we typically make some assumptions for simplification. Each user is assumed to have one single-threaded program where each program is independent of each other.

Goals and Criteria

The goal of scheduling is to dole out CPU time to optimize some desired parameters of the system. Generally speaking, scheduling policies focus on the following goals and criteria.

Minimize Completion Time

Completion time is the combination of the waiting time plus the run time of a process (e.g. time to compile a program, time to echo a keystroke in an editor). Minimizing completion time is crucial for time-sensitive tasks (e.g. I/O).

Maximize Throughput

Throughput is the rate at which tasks are completed. While throughput is related to completion time, they are not the same. Maximizing throughput involves minimizing overhead (e.g. context switching) and efficient use of resources.

Maintain Fairness

Fairness refers to sharing resources in some equitable manner. Evidently, fairness is not very well defined unless specific parameters are specified. Maintaining fairness will usually contradict minimizing completion time.

First Come First Serve (FCFS)

First come first serve (FCFS) schedules tasks in the order in which they arrive. It's simple to implement and is good for throughput since it minimizes the overhead of context switching. However, average completion time under FCFS can very significantly according to arrival order. Additionally, FCFS suffers from the **Convoy effect** where short tasks get stuck behind long tasks.

Shortest Job First (SJF) / Shortest Remaining Time First (SRTF)

Shortest job first (SJF) schedules the shortest task first. **Shortest remaining time first (SRTF)** is a preemptive version of SJF. If a task arrives and has a shorter time to completion than the current running task, the resource will be preempted. SJF and SRTF are provably optimal for minimizing average completion time among non-preemptive and preemptive schedulers, respectively. However, both of them involve the impossible idea of knowing how long a task is going to take.

Moreover, they do not maintain fairness with regards to the amount of time spent using a resource. In fact, SJF and SRTF can lead to **starvation** which is the continued lack of progress for one task due to other tasks being scheduled over it. If small tasks keep arriving, larger tasks will never get to run.

Round Robin (RR)

Round robin (RR) schedules tasks such that each of them takes turns using a resource for a small unit of time known as the **time quantum** (q). After q expires, the task is preempted and added to the end of the ready queue. When q is large, RR resembles FCFS, becoming identical if q is larger than the length of

any task. When q is small, RR resembles SJF Generally, q must be large with respect to the cost of context switching to avoid the overhead being too high.

If there are n tasks, each task gets $1/n$ amount of resources, ensuring fairness in terms of sharing resources. No task will wait for more than $(n-1)q$ time units.

In general, with RR, small scheduling quantum decreases average waiting time but increases completion time, due to the extra context switching overhead and the fact that longer jobs get "stretched out".

Lottery

Lottery gives each task a certain number of lottery tickets. On each time slice, a ticket is randomly picked. On expectation, each task will be given time proportional to the number tickets it was originally given. When distributing tickets, it's important to make sure that every task gets at least one ticket to avoid starvation. If SRTF was to be approximated, shorter tasks would simply get more tickets than longer tasks.

Multi-Level Feedback Queue (MLFQ)

Multi-level feedback queue (MLFQ) uses multiple queues which each have different priority. Each queue has its own scheduling algorithm. Higher-priority queues (foreground) will typically use RR while lower-priority queues (background) might use FCFS.

A task will start at the highest-priority queue. If the task uses up all the resources (e.g. q for RR), it will get pushed one level down as a penalty. If the task does not use up all the resources (e.g. less than q for RR), it will get pushed one level up as a reward. This ensures that long running tasks (e.g. CPU bound) don't hog all resources and get demoted into low priority queues while short running tasks (e.g. I/O bound) will remain at the higher-priority queues.

1.1 Round Robin T/F

1. The average wait time is less than that of FCFS for the same workload.

   False. This is generally not true when the time quantum is small. Consider tasks A, B, and C (arriving in this order) which each take 3 time units. With FCFS, the average wait time is 3. With RR ($q = 1$), average wait time is 5.

2. If a quantum is constantly updated to become the number of cpu ticks since boot, Round Robin becomes FCFS.

   True. Quantum never gets used for any task since it always increases as the task progresses.

3. Ideally, you should set the time quanta of round robin generally to be significantly smaller than the average CPU burst time of a task.

   False. This would result in poor turnaround time, since tasks will have to wait through multiple cycles for their turns. Ideally, you'd want the quanta to be large enough to absorb the CPU burst of most tasks, but small enough to prevent short-running tasks from being starved by long ones.

4. Cache performance is likely to improve relative to FCFS.

   False. RR usually results in more context switches when compared to FCFS, meaning the cache will have more misses.

5. If no new threads are entering the system, all threads will get a chance to run in the cpu every `QUANTA*SECONDS_PER_TICK*NUMTHREADS` seconds, assuming `QUANTA` is in ticks.

   False. There exists context switching overhead.

1.2 Life Ain't Fair

Suppose the following threads (**priorities given in parentheses**) arrive in the ready queue at the clock ticks shown. Assume all threads arrive unblocked and that **each takes 5 clock ticks to finish executing**. Assume threads arrive in the queue at the beginning of the time slices shown and are ready to be scheduled in that same clock tick. This means you update the ready queue with the arrival before you schedule/execute that clock tick. Assume you only have one physical CPU.

```
0   Taj (prio = 7)
1
2   Kevin (prio = 1)
3   Neil (prio = 3)
4
5   Akshat (prio = 5)
6
7   William (prio = 11)
8
9   Alina (prio = 14)
```

Determine the order and time allocations of execution for each given scheduler scenario. Write answers in the form of vertical columns with one name per row, each denoting one clock tick of execution. For example, allowing Taj 3 units at first looks like:

```
0   Taj
1   Taj
2   Taj
```

It will probably help you to draw a diagram of the ready queue at each tick for this problem.

1. Round robin with time quantum 3

   From t=0 to t=3, Taj gets to run since there is initially no one else on the run queue. At t=3, Taj gets preempted since the time slice is 3. Kevin is selected as the next person to run, and Neil gets added to the run queue just before Taj. Kevin is the next person to run from t=3 to t=6. At t=5, Akshat gets added to the run queue, which consists of at this point: Neil, Taj, Akshat At t=6, Kevin gets preempted and Neil gets to run since he is next. Kevin gets added to the back of the queue, which consists of: Taj, Akshat, Kevin. From t=6 to t=9, Neil gets to run and then is preempted. Taj gets to run again from t=9 to t=10, and then finishes executing. Akshat gets to run next and this pattern continues until everyone has completed running

0	Taj	10	Taj	20	Alina
1	Taj	11	Akshat	21	Alina
2	Taj	12	Akshat	22	Neil
3	Kevin	13	Akshat	23	Neil
4	Kevin	14	Kevin	24	Akshat
5	Kevin	15	Kevin	25	Akshat
6	Neil	16	William	26	William
7	Neil	17	William	27	William
8	Neil	18	William	28	Alina
9	Taj	19	Alina	29	Alina

2. SRTF with preemptions

Since each thread takes 5 ticks, a thread will never be preempted once begun. As a result, each thread will just execute in order of arrival (i.e. FIFO).

0	Taj	10	Neil	20	William
1	Taj	11	Neil	21	William
2	Taj	12	Neil	22	William
3	Taj	13	Neil	23	William
4	Taj	14	Neil	24	William
5	Kevin	15	Akshat	25	Alina
6	Kevin	16	Akshat	26	Alina
7	Kevin	17	Akshat	27	Alina
8	Kevin	18	Akshat	28	Alina
9	Kevin	19	Akshat	29	Alina

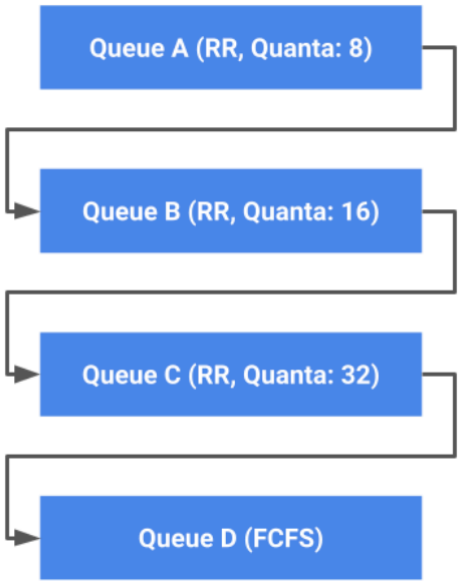
3. Preemptive priority

Even though Kevin and Neil arrive while Taj is running, Taj has higher priority, so it runs to completion. Just as Taj finishes, Akshat arrives with a higher priority than Kevin and Neil who are waiting, so Akshat runs for two clock ticks until William arrives with a greater priority. Similarly, William runs for two clock ticks until Alina arrives with a greater priority. Alina runs to completion, then the remaining threads each run to completion based on priority.

0	Taj	10	Alina	20	Neil
1	Taj	11	Alina	21	Neil
2	Taj	12	Alina	22	Neil
3	Taj	13	Alina	23	Neil
4	Taj	14	William	24	Neil
5	Akshat	15	William	25	Kevin
6	Akshat	16	William	26	Kevin
7	William	17	Akshat	27	Kevin
8	William	18	Akshat	28	Kevin
9	Alina	19	Akshat	29	Kevin

1.3 Bitcoin Mining

You are a Bitcoin miner, and you've developed an algorithm that can run on an unsuspecting machine and mine Bitcoin. You now need to write a program that will run your mining algorithm forever. While you want your mining job to be scheduled often, you also don't want to attract too much suspicion from system users or administrators. Fortunately, you know that the machines you're targeting use a MLFQS algorithm to schedule jobs, outlined below.



1. You decide that the best strategy is to guarantee that your mining job will always be placed on Queues B and C.

   Assume that the CPU-intensive mining algorithm you've developed can be run in 10-tick intervals. Implement your mining program, and explain your design. The only functions you should use are `mine` (which runs for 10 ticks) and `printf`. Assume that your job is initially placed on Queue B.

```c
void mine_forever() {
  while(1) {
    ---------------
    ---------------
    ---------------
  }
}
```

bitcoin_mine.c

```c
void mine_forever() {
  while(1) {
    for (int i = 0; i < 4; i++)
      mine();
    printf("Not a Bitcoin miner!!!");
  }
}
```

bitcoin_mine_sol.c

The program needs to exceed the Queue B quanta so that the quanta expires and the job is

placed on Queue C. Once placed on Queue C, the job needs to voluntarily yield so that it can be placed on Queue B once again. By running the mining algorithm 4 times, the first 2 iterations will cause the Queue B quanta to expire. After the remaining 2 iterations, the job should voluntarily yield (e.g. print to stdout). Note: Once the job is placed on Queue C, it will still run the 2nd iteration of the mining algorithm for 4 ticks.

2. Explain why, regardless of how you implement your mining program, your job will never be placed on Queue A twice in a row.

Since the mining algorithm can only be run in 10 tick intervals, any implementation will always exceed the Queue A quanta before the CPU can be voluntarily yielded. This will cause the job to be placed on Queue B, since the Queue A quanta expired.

2 Starvation

When designing scheduling policies, it's important to prevent **starvation**, a situation where a thread fails to make progress for an indefinite period of time. If a scheduling policy never runs a particular thread on the CPU, the thread will starve. Threads can also starve if they wait for each other or are spinning in a way that will never be resolved.

Strict Priority

A **strict priority scheduler** schedules the task with the highest priority. If multiple tasks have the same priority, they can be scheduled in some RR fashion. This ensures that important tasks get to run first but doesn't maintain fairness. Evidently, a strict priority scheduler suffers from starvation for lower priority threads.

Priority Inversion

With a priority scheduler (e.g. strict priority), **priority inversion** can become a problem where a higher priority task is blocked waiting on a lower priority task. As a result, a medium priority task (in between the higher and lower priority) will be run by the scheduler, resulting in the medium priority task starving the higher priority task. A common example of this is when a higher priority task tries to acquire a mutex that the lower priority task already holds.

To address priority inversion, the scheduler can use **priority donation** where the lower priority task is *temporarily* granted the same priority as the higher priority task, so it can run. In this scenario, the lower priority thread is said to have an **effective priority** of the higher priority task. Once the higher priority task is no longer blocked on the lower priority task, the scheduler would demote it back to its **base priority**.

Lottery

A **lottery scheduler** gives each task some number of lottery tickets. At each time slice, a random ticket is drawn, and the task holding that ticket is granted the resource. On expectation, the time each task is allocated the resource for will be proportional to the number of tickets it holds.

Ticket numbers can be assigned in a variety of schemes. For instance, the scheduler could approximate SRTF by giving more tickets to shorter jobs and less tickets to longer jobs. Essentially, the number of tickets serve as a measure of priority in some sense. To avoid starvation, it's important to make sure that every job gets at least one ticket.

Stride

A **stride scheduler** is a deterministic version of a lottery scheduler. Like a lottery scheduler, the stride scheduler gives each task some number of tickets. Then, each task is defined a **stride** which is inversely proportional to the number of tickets. Typically, this is calculated as W/n_i where W is a really big number and n_i is the number of tickets given to task i.

On every time slice, the task with the lowest **pass** is chosen. All tasks start with **pass** equal to 0. When a task is chosen, its pass is incremented by its stride. Hence, a smaller stride will allow a task to run more often.

Linux Completely Fair Scheduler (CFS)

The **Linux Completely Fair Scheduler (CFS)** aims to give each task an equal share of the CPU by giving an illusion that each task executes simultaneously on $1/n$ of the CPU. Since hardware needs to give out CPU in full time slices, the scheduler will track CPU time per task and schedule tasks to match up the average rate of execution. When choosing a task to run, the thread with minimum CPU time will be chosen. To accomplish this efficiently, a heap like scheduling queue is used for efficient (logarithmic with respect to number of tasks) popping and pushing operations.

Deadlock

Deadlock is a situation where there is a cycle of waiting among a set of threads, where each thread waits for some other thread in the cycle to take some action. Deadlock is a special form of starvation but has a stronger condition since starvation can end but deadlock cannot.

There are four necessary *but not sufficient* conditions for a deadlock to occur. Eliminating any one of these will eliminate the possibility of a deadlock.

Mutual Exclusion and Bounded Resources
Finite number of threads (usually one) can simultaneously use a resource.

Hold and Wait
Thread holds one resource while waiting to acquire additional resources held by other threads.

No Preemption
Once a thread acquires a resource, its ownership cannot be revoked until the thread acts to release it.

Circular Waiting
There exists a set of waiting threads such that each thread is waiting for a resource held by another.

Detection

The following is a deadlock detection algorithm.

Require:
available, array of how much of each resource is available
alloc, 2D array where the i-th element is how many of each resource thread i currently holds.
request, 2D array where the i-th element is the how many of each of resource thread i is requesting.

```
unfinished ← all threads
done ← false
while done is false do
  done ← true
  for thread in unfinished do
    if request[thread] ≤ available then
      remove thread from unfinished
      available ← available + alloc[thread]
      done ← false
    end if
  end for
end while
```

When the algorithm finishes, the system is said to be deadlocked if there are threads left in unfinished.

Handling

There are four main ways a system can handle a deadlock.

Denial
Pretend deadlock is not a problem (i.e. ostrich algorithm). This is usually effective when deadlocks are uncommon. In the rare times when deadlocks do occur, the system is usually restarted.

Prevention
Write systems that don't result in deadlock. This typically involves eliminating one of the four necessary conditions. Infinite resources could eliminate the bounded resources problem. While not possible for all types of resources, an illusion of infinite resources typically suffices (e.g. virtual memory). No sharing of resources can eliminate mutual exclusion, but totally independent threads are not very realistic and defeats the purpose of using threads. Forcing all threads to request resources in a particular order can also be useful and typically done in many scenarios, but it does require careful coding practices.

Threads could also be forced to not wait and instead keep trying until a successful acquisition of a resource, which is quite inefficient.

Recovery

Let deadlock happen, and recover from it afterwards. A simple and intuitive approach may be to terminate a thread, forcing it to give up its resources. However, this isn't always possible since killing a thread holding a mutex leaves the system in an inconsistent state. The system could also try to take away resources from the thread temporarily, but that may be difficult to fit the semantics of computation. A more general technique is to roll back actions of deadlocked threads. However, this is also prone to deadlock in the same way again if threads are executed in the same order.

Avoidance

Dynamically delay resource requests, so deadlock doesn't happen. The first idea that comes to mind is to check if a resource request would result in a deadlock when a thread requests a resource. However, this may be too late as the system may already be in a **unsafe state** where there isn't a deadlock yet but there is potential for a pattern of resource requests that unavoidably leads to deadlock. The system must always be kept in a **safe state** where the system can delay resource acquisition requests to prevent deadlock. As a result, the system needs to check if a resource request would result in an *unsafe* state not a deadlocked state.

Banker's algorithm is a deadlock avoidance technique. A thread states the max amount of each resource it needs in advance. The conservative approach would be to allow a particular thread to proceed if the available resources - number requested is at least the max number needed by any other thread. Banker's algorithm takes a less conservative route and pretends each request is granted. Then, it runs the deadlock detection algorithm with an updated condition as seen below.

Require:
available, array of how much of each resource is available
alloc, 2D array where the i-th element is how many of each resource thread i currently holds.
max, 2D array where the i-th element is the max number of resources thread i will request.

```
unfinished ← all threads
done ← false
while done is false do
  done ← true
  for thread in unfinished do
    if max[thread] - alloc[thread] ≤ available then
      remove thread from unfinished
      available ← available + alloc[thread]
      done ← false
    end if
  end for
end while
```

The key idea is that banker's algorithm determines if threads are able to be scheduled in a way to avoid deadlock. However, not every execution order has to avoid deadlock.

2.1 Concept Check

1. In what sense is Linux CFS completely fair?

   CFS attempt to give all tasks equal access to the CPU.

2. How can you easily implement lottery scheduling?

> Given that each task is allocated their own set number of tickets totaling N tickets across all threads, we can select at random a number between 1 through N. This number would fall into a bin defined by the number of tickets per thread and that thread would then get to run.

3. Is stride scheduling prone to starvation?

> No. Stride scheduling tries to achieve proportional shares to the CPU so a thread will eventually get a chance at running. Concretely, if all other threads' pass value are strictly increasing, then even the lowest priority thread will get a chance at running.

4. When using stride scheduling, if a task is more urgent, should it be assigned a larger stride or a smaller stride?

> Smaller stride. Lower stride tasks run more often as their pass value increases at a slower rate.

2.2 Simple Priority Scheduler

Let's implement a new scheduler in Pintos called the simple priority scheduler (SPS). We will just split threads into two priorities: high (1) and low (0). High priority threads should always be scheduled before low priority threads. Turns out we can do this without expensive list operations.

```
struct thread {
  ...
  int priority;
  struct list_elem elem;
  ...
}

struct list ready_list;

void thread_unblock (struct thread *t)  {
  ASSERT(is_thread(t));

  enum intr_level old_level;
  old_level = intr_disable();
  ASSERT(t->status == THREAD_BLOCKED);

  if (__________) {
    ____________________;
  } else {
    ____________________;
  }

  t->status = THREAD_READY;
  intr_set_level(old_level);
}
```

pintos_sps.c

1. Complete the blanks of `thread_unblock` to complete implement SPS. Assume that SPS will treat the ready queue as FIFO. You may ignore any possibilities of preemptions.

> If the current thread has a high priority, we place it at the front of the ready queue. Otherwise, we place it at the back.

```
void thread_unblock (struct thread *t)  {
  ASSERT(is_thread(t));

  enum intr_level old_level;
  old_level = intr_disable();
  ASSERT(t->status == THREAD_BLOCKED);

  if (t->priority == 1) {
    list_push_front(&ready_list, &t->elem);
  } else {
    list_push_back(&ready_list, &t->elem);
  }

  t->status = THREAD_READY;
  intr_set_level(old_level);
}
```

pintos_sps_sol.c

2. In order for this scheduler to be "fair" briefly describe when you would make a thread high priority and when you would make a thread low priority.

   Downgrade priority when thread uses up its quanta, upgrade priority when it voluntarily yields, or gets blocked.

3. If we let the user set the priorities of this scheduler with `set_priority`, why might this scheduler be preferable to the normal pintos priority scheduler?

   The insert operations are cheaper, and it provides a good approximation to priority scheduling.

4. How can we trade off between the coarse granularity of SPS and the super fine granularity of normal priority scheduling? Assume we still want a fast insert.

   We can have more than 2 priorities but still a small number of fixed priorities, and have a queue for each priority, and then pop off threads from each queue as necessary.

2.3 Banker's Algorithm

Suppose we have the following resources: A, B, C and threads T1, T2, T3 and T4. The total number of each resource as well as the current/max allocations for each thread are as follows:

Total		
A	B	C
7	8	9

	Current			Max		
T/R	A	B	C	A	B	C
T1	0	2	2	4	3	3
T2	2	2	1	3	6	9
T3	3	0	4	3	1	5
T4	1	3	1	3	3	4

1. Is the system in a safe state? If so, show a non-blocking sequence of thread executions.

Yes. To find a safe sequence of executions, we need to first calculate the available resources and the needed resources for each thread. To find the available resources, we sum up the currently held resources from each thread and subtract that from the total resources.

Available		
A	B	C
1	1	1

To find the needed resources for each thread, we subtract the resources they currently have from the maximum they need.

Needed			
	A	B	C
T1	4	1	1
T2	1	4	8
T3	0	1	1
T4	2	0	3

From these, we see that we must run T3 first, as that is the only thread for which all needed resources are currently available. After T3 runs, it returns its held resources to the resource pool, so the available resource pool is now as follows.

Available		
A	B	C
4	1	5

We can now run either T1 or T4, and following the same process, we can arrive at a possible execution sequence of either $T3 \rightarrow T1 \rightarrow T4 \rightarrow T2$ or $T3 \rightarrow T4 \rightarrow T1 \rightarrow T2$.

2. If the total number of C instances is 8 instead of 9, is the system still in a safe state?

Following the same procedure from the previous question, we see that there are 0 instances of C available at the start of this execution. However, every thread needs at least 1 instance of C to run, so we are unable to run any threads and thus the system is not in a safe state.

Problem 3: Atomic Synchronization and the Fair Lock [30pts]

In class, we discussed a number of *atomic* hardware primitives that are available on modern architectures. In particular, we discussed "test and set" (TSET), SWAP, and "compare and swap" (CAS). They can be defined as follows (let "expr" be an expression, "&addr" be an address of a memory location, and "M[addr]" be the actual memory location at address addr):

Test and Set (TSET)	Atomic Swap (SWAP)	Compare and Swap (CAS)

`TSET(&addr) {`
`  int result = M[addr];`
`  M[addr] = 1;`
`  return (result);`
`}`
	`SWAP(&addr, expr) {`
	`  int result = M[addr];`
	`  M[addr] = expr;`
	`  return (result);`
	`}`
		`CAS(&addr, expr1, expr2) {`
		`  if (M[addr] == expr1) {`
		`  M[addr] = expr2;`
		`  return true;`
		`  } else {`
		`  return false;`
		`  }`
		`}`

Both TSET and SWAP return values (from memory), whereas CAS returns either true or false. Note that our &addr notation is similar to a reference in c, and means that the &addr argument must be something that can be stored into. For instance, TSET could be used to implement a spin-lock acquire as follows:

```
int lock = 0; // lock is free

// Later: acquire lock
while (TSET(&lock));
```

Problem 3a[2pts]: Explain what it means for these operations to be atomic. Why is this behavior desirable enough that all modern processors after MIPS have some variation of the above operations? *Please answer in 3 sentences or less.*

What it means for these operations to be "atomic" is that they cannot be interleaved with other instructions or operations, i.e. at any given time, either all of the subcomponents occur as a unit or none of them do. These operations are typically implemented in hardware as single instructions. The existence of these operations is important for synchronization at user level: without them (such as with MIPS), the only other option for synchronizing between user-level threads is to do something really complex/non-symmetric with load/store operations such as with the "Got Milk" example from class.

Problem 3b[2pts]: Show how to implement a spinlock `acquire()` with a single while loop using CAS instead of TSET. Fill in the arguments to CAS below. *Hint: need to wait until can grab lock.*

```
void acquire(int *mylock) {

  while (!CAS(mylock, ___0_____________, ___1_____________ ));
}
```

As discussed in class, `Futex()` is a *system call* that allows a thread to *put itself to sleep,* under certain circumstances, on a queue associated with a user-level address. It also allows a thread to ask the kernel to wake up threads that might be sleeping on the same queue. Recall that the function signature for `Futex` is:

```
int futex(int *uaddr, int futex_op, int val);
```

In this problem, we focus on two `futex_op` values:

1. For `FUTEX_WAIT`, the kernel checks atomically as follows:

   if (`*uaddr == val`): the calling thread will sleep and another thread will start running
   if (`*uaddr != val`): the calling thread will keep running, i.e. `futex()` returns immediately
2. For `FUTEX_WAKE`, this function will wake up to val waiting threads.

Problem 3c[2pts]: Fill out the missing blanks, in the `acquire()` function, below, to make a version of a lock that does not busy wait. The corresponding `release()` function is given for you:

```
void acquire(int *thelock) { // Acquire a lock
   while (TSET(thelock)) {

      futex(thelock, ___FUTEX_WAIT_____, ____1_____________);
   }
}
void release(int *thelock) { // Release a lock
   *thelock = 0;
   futex(thelock, FUTEX_WAKE, 1);
}
```

In the remainder of this problem, we will develop a queue-lock version of `acquire()` and `release()` that has the following properties:

1. Threads will be granted access to the lock in FIFO order, based on the time that they enter `acquire()`. The order of threads entering simultaneously will be undefined.
2. Threads that are waiting on the lock will be put to sleep (*not spinning).* So, there should be no spin waiting at all in this problem!

To build a queue, we will need to link waiting threads together somehow, which will require individual links in our queue. To make this interesting, we will build our queue lock without calling malloc() or free(), but rather using stack-allocated local structures that are automatically released when a procedure exits. Note that we will be building a FIFO queue in which the head of the queue is attached to the "anchor" element, and in which items are added to the tail. Here are the definitions we will use to make this work:

```
// Possible lock states
typedef enum {
   UNLOCKED, LOCKED
} LockState;

// The actual structure of a queue lock!  Every lock has one of these.
typedef struct QueueLock {
   LockEntry *tail;
   LockEntry anchor;
   LockState lockvar;
} QueueLock;

// Every linked thread has one of these structures while it is waiting.
typedef struct LockEntry {
   LockEntry *next;
   int waitlock;
   int waitnext;
} LockEntry;

// This is our actual queue:
QueueLock ourLock;
```

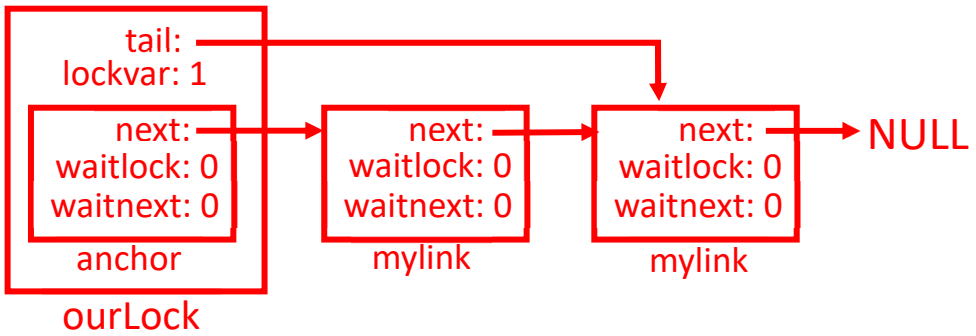
Problem 3d[2pts]: Fill out the following lines to complete the initialization for the queue. Remember – you are trying to initialize this queue as *empty.* Note that we have already added the semicolons, so you shouldn't need any additional ones. Hint: a structure element of type "`struct foo`" can be initialized to all zero with "`foo = {};`"

```
void LockInit(QueueLock *myLock) {

   myLock->tail = &(myLock->anchor);                    ;

   myLock->anchor = {};                                 ;

   myLock->Lockvar = UNLOCKED;
}
```

Now, *ignoring concurrency* for a moment, our lock `acquire()` and `release()` routines are going to look like the following code sketch. We have numbered code lines for reference in later problems. Note that the acquire version spins on its own local `waitlock` entry, which means that any shared memory traffic generated by it will not interfere with the spinning of any other threads:

```
    void acquire(QueueLock *myLock) {
1:     LockEntry mylink = {};       // Stack allocated queue link

       // Get our position in line by enqueuing us at the end of the list
2:     LockEntry *oldtail = myLock->tail;
3:     myLock->tail = &mylink;
4:     oldtail->next = &mylink;

       // Wait for lock to become free
5:     If (myLock->anchor.next != &mylink || myLock->lockvar == LOCKED) {
6:        while (!mylink.waitlock);
7:     }
8:     myLock->lockvar = LOCKED;

       // dequeue ourself from the queue.  We know we are at head of queue
9:     if (myLock->tail == &mylink) {
10:       myLock->tail = &(myLock->anchor);
11:       myLock->anchor.next = NULL;
12:       return;
13:    }
14:    myLock->anchor.next = mylink.next
15:    return;
    }

    // Release – either give to next thread in line, or release lock
    void release(QueueLock *myLock) {
17:    if (myLock->tail != &(myLock->anchor)) {
          // Wake up the thread
18:       myLock->anchor.next->waitlock = 1;
19:    } else {
20:       myLock->lockvar = UNLOCKED;
21:    }
    }
```

Problem 3e[2pts]: Draw a picture of the structure of the queue after 2 items are linked into it. Use a box for each LockEntry item, and arrows for pointers:



Problem 3f[3pts]: For each of the following potential context switch points, state whether or not a context switch at that point could cause incorrect behavior. List the potential issues with switching at that point. (Each point may have multiple issues.) Choose from: ***No Problem, Bad Enqueue, Bad Dequeue, Bad Release.*** Explain your choice.

```
                void acquire(QueueLock *myLock) {
Point 1 ------>     LockEntry mylink = {};
Point 2 ------>     LockEntry *oldtail = myLock->tail;
Point 3 ------>     myLock->tail = &mylink;
                    oldtail->next = &mylink;
```

Point 1: *No Problem. At this point, there is no issue with switching and returning, since no global state has been altered yet.*

Point 2: *Bad Enqueue. At this point, if we switch out and return, the tail pointer (myLock->tail) may have changed, so we will link things incorrectly.*

Point 3: *Bad Dequeue/Bad Release. At this point, if we switch out, we will have successfully made ourselves visible as the tail of the list. However, we haven't updated the previous element to point at us, so dequeueing will fail and/or an attempt by the previous lock holder to inform us during release() will fail.*

Problem 3g[3pts]: Although you cannot fix all of the problems you identified in problem **(3f)**, you can at least make sure that every `acquire()` operation properly enqueues the threads on the FIFO queue, thereby choosing the final order of lock acquisition regardless of the concurrency of the operations. We say that the `acquire()` operations are properly serialized in those circumstances. Rewrite lines 1-4 to guarantee proper serialization (*HINT: you will use a* `do/while` *loop with CAS*). We started you off with the first two lines:

```
    void acquire(QueueLock *myLock) {
1:      LockEntry mylink = {};
A:      do {
B:          *oldtail = myLock->tail;
C:      } while(!CAS(&(myLock->tail), oldtail, &mylink);
D:      oldtail->next = &mylink
```

Note: The C language doesn't allow us to declarare `oldtail` on line B because it goes out of scope in the "while" condition. So, ideally, we would have another line before "do" to declare "`LockEntry *oldtail;`" We will not penalize someone who does declare "LockTail `*oldtail=`" here, however!

Problem 3h[4pts]: Why is the dequeue code at lines 9-13 problematic when concurrent threads are trying to `acquire()` the queue lock? Explain the problem and produce an alternative solution to fix this problem in the case of concurrency with acquire(). Note that you are coming up with code that will *replace* lines 9-13. You do not have to use every line here, and can do this with TWO semicolons. *Hint: make sure to think about concurrent switch points with new `acquire()` requests.*

```
       // Original version of lines 9-13
9:     if (myLock->tail == &mylink) {
10:       myLock->tail = &(myLock->anchor);
11:       myLock->anchor.next = NULL;
12:       return;
13:    }
```

Explain Issue: *The issue here is that things might change after we decide that we are last on queue (in Line 9). Two issues: (1) we might delete new queue entry by accident if there is a switch between Lines 9 and 10 or (2) we might destroy link from anchor to new entry if there is a switch between Lines 10 and 11.*

Note: Our solution will be clever here in that we know before Line 9 that we are pointed at by `myLock->anchor.next` and can clear that link, but later fix it in Line 14+ if we aren't last in queue.

```
       // Thread-safe version of original lines 9-13
E:     myLock->anchor.next = NULL;  // will fix in Line 14 if needed
F:     if (CAS(myLock->tail, &mylink, &(myLock->anchor))
G:         return;
H:
```

Problem 3i[4pts]: The `acquire()` code busywaits threads waiting for the lock. Identify the line responsible for busywaiting, and fix the code to remove busywaiting. You do not need to use every line, below, in your updated version of `release()`. *Hint: be careful because there is a race condition in release() as well, but it is easy to fix – think back to your answer in problem* ***(3g)***.

Line # to replace: *6*
What should you replace it with:

```
I:   futex(mylink->waitlock, FUTEX_WAIT, 0);
```

```
   // Original release code
   void release(QueueLock *myLock) {
17:    if (myLock->tail != &(myLock->anchor)) {
          // Wake up the thread
18:       myLock->anchor.next->waitlock = 1;
19:    } else {
20:       myLock->lockvar = UNLOCKED;
21:    }
   }
```

Updated version of the release code:

```
   void release(QueueLock *myLock) {
J:    if (myLock->tail != &(myLock->anchor)) {
K:       If (mylink->anchor.next != NULL) {
L:          myLock->anchor.next->waitlock = 1;
M:          futex(myLock->anchor.next.waitlock,FUTEX_WAKE, 1);
N:       }
19:    } else {
20:       myLock->lockvar = UNLOCKED;
21:    }
   }
```

Problem 3j[2pts]: The code in lines 14-15 is problematic when concurrent threads are trying to `acquire()` the queue lock. Explain what the problem is and how you need to fix it from a high level. You will write actual code in problem **(3k)**, below. *Hint: think about premature freeing of objects on the stack. Your fix will likely involve waiting (notice unused element in `LockEntry`).*

```
14:    myLock->anchor.next = mylink.next
15:    return;
```

Explain Issue: *After we return (i.e. Line 15), the `mylink` structure will be freed, since the stack frame is destroyed. We cannot do that until we are sure that `mylink` is not going to be used by anbother thread. By Line 14, we know that some other LockEntry structure is either already fully linked to us or in the process of being linked (because of Lines 9-13). If `mylink.next == NULL`, we know that some other thread will still set it., so we must wait until `mylink.net != NULL`. We will use `LockEntry->waitnext` as a signal.*

Problem 3k[4pts]: Combine everything you have learned to make working code for `acquire()`. You have already given the new version of release() in problem **(3k)**. In the following, you may repeat code from the original solution by naming the line or a range of lines without rewriting them. For instance, if you wanted to repeat line 1 you would just say "`Line 1`". Or, if you wanted to repeat lines 1-4 you would just say "`Lines 1-4`". For all the other lines, you should fill them. You are also allowed to reference solutions to problems **(3g)** – **(3i)** with lines A-I.

Make sure to use *ALL* of lines 1-21 except those that were replaced by your answers to problems **(3g)** – **(3i)**. We started you out with 5 lines. You are not required to use all of the spaces:

```
void acquire(QueueLock *myLock) {
    Line 1
    Lines A-D
    if (oldtail->waitnext)
        futex(&oldtail->waitnext, FUTEX_WAKE, 1);
    Line 5
    Line I
    Lines 7-8
    Lines E-G
    if (mylink->next) {
        mylink->waitnext = 1;
        futex(&mylink->waitnext, FUTEX_WAIT, 0);
    }
    Lines 14-15
}
```